

Like and Comment Functionality Implementation Guide

Overview

This implementation adds like and comment functionality to community posts in The Writer's Corner application. Users can now:

- Like and unlike posts (toggle functionality)
- Add comments to posts
- Edit their own comments
- Delete their own comments
- View like and comment counts in real-time

Changes Made

1. Database Schema Updates (`app/prisma/schema.prisma`)

Added two new models:

- **PostLike**: Stores user likes on posts
- Unique constraint on (`postId`, `userId`) to prevent duplicate likes
- Cascade delete when post or user is deleted
- **PostComment**: Stores user comments on posts
- Supports content, timestamps, and user association
- Cascade delete when post or user is deleted

Updated existing models:

- **User**: Added `postLikes` and `postComments` relations
- **ExerciseSubmission**: Added `likes` and `comments` relations (since community posts are based on exercise submissions)

2. Backend API Endpoints

Created the following API routes:

Like Functionality

- **POST** `/api/community/posts/[postId]/like` : Toggle like on a post
- **GET** `/api/community/posts/[postId]/like` : Get like status and count for a post

Comment Functionality

- **GET** `/api/community/posts/[postId]/comments` : Get all comments for a post
- **POST** `/api/community/posts/[postId]/comments` : Add a new comment
- **PATCH** `/api/community/comments/[commentId]` : Edit a comment (only by author)
- **DELETE** `/api/community/comments/[commentId]` : Delete a comment (only by author)

Updated Existing Endpoint

- **GET** `/api/community/posts/route.ts` : Enhanced to include like counts, comment counts, and user's like status

3. Frontend Components

Created new React components:

PostLikeButton (`app/components/community/post-like-button.tsx`)

- Displays like count and current user's like status
- Toggleable heart icon (filled when liked)
- Optimistic UI updates for better UX
- Error handling with rollback

PostComments (`app/components/community/post-comments.tsx`)

- Displays comment count and toggle to show/hide comments
- Comment form for adding new comments
- Edit and delete buttons for user's own comments
- Relative timestamps (e.g., "2 hours ago", "Yesterday")
- Real-time comment count updates

Updated CommunityOverview

- Integrated new like and comment components
- Updated to handle new post data structure

4. Authentication Integration

- All API endpoints use NextAuth session authentication
- Users must be logged in to like, comment, or view posts
- Edit and delete operations restricted to comment authors

Setup Instructions

1. Install Dependencies

```
cd app
npm install
```

2. Generate Prisma Client

```
npx prisma generate
```

3. Create Database Migration

```
npx prisma migrate dev --name add-likes-and-comments
```

This will:

- Create the migration SQL files
- Apply the migration to your database
- Generate the updated Prisma Client

4. Run the Application

```
npm run dev
```

Visit `http://localhost:3000/community` to see the new functionality in action.

Database Migration Details

The migration adds the following tables:

```
-- PostLike table
CREATE TABLE "PostLike" (
  "id" TEXT NOT NULL PRIMARY KEY,
  "postId" TEXT NOT NULL,
  "userId" TEXT NOT NULL,
  "createdAt" TIMESTAMPT(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY ("postId") REFERENCES "ExerciseSubmission"("id") ON DELETE CASCADE,
  FOREIGN KEY ("userId") REFERENCES "User"("id") ON DELETE CASCADE
);

CREATE UNIQUE INDEX "PostLike_postId_userId_key" ON "PostLike"("postId", "userId");
CREATE INDEX "PostLike_postId_idx" ON "PostLike"("postId");
CREATE INDEX "PostLike_userId_idx" ON "PostLike"("userId");

-- PostComment table
CREATE TABLE "PostComment" (
  "id" TEXT NOT NULL PRIMARY KEY,
  "content" TEXT NOT NULL,
  "postId" TEXT NOT NULL,
  "userId" TEXT NOT NULL,
  "createdAt" TIMESTAMPT(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMPT(3) NOT NULL,
  FOREIGN KEY ("postId") REFERENCES "ExerciseSubmission"("id") ON DELETE CASCADE,
  FOREIGN KEY ("userId") REFERENCES "User"("id") ON DELETE CASCADE
);

CREATE INDEX "PostComment_postId_idx" ON "PostComment"("postId");
CREATE INDEX "PostComment_userId_idx" ON "PostComment"("userId");
```

Features

Like Functionality

- **Toggle Behavior:** Click to like, click again to unlike
- **Visual Feedback:** Filled heart icon when liked, outline when not liked
- **Real-time Counts:** Like count updates immediately
- **Optimistic Updates:** UI updates instantly, with rollback on error

Comment Functionality

- **Add Comments:** Multi-line text area with validation
- **Edit Comments:** Only for comment authors, preserves timestamps
- **Delete Comments:** With confirmation dialog
- **Relative Timestamps:** Human-readable time display
- **Responsive UI:** Matches existing vintage/typewriter design

Design Decisions

1. Database Design:

- Used separate tables for likes and comments for better scalability
- Added indexes on frequently queried fields (postId, userId)
- Unique constraint on likes prevents duplicate entries
- Cascade deletes maintain referential integrity

2. API Design:

- RESTful endpoints for clear resource management
- Consistent error handling and status codes
- Session-based authentication using NextAuth

3. Frontend Design:

- Component-based architecture for reusability
- Optimistic UI updates for better perceived performance
- Error handling with user-friendly toast notifications
- Design matches existing vintage/typewriter aesthetic

Testing

Manual Testing Checklist

- ☐ Like a post and verify count increases
- ☐ Unlike a post and verify count decreases
- ☐ Add a comment and verify it appears
- ☐ Edit own comment and verify changes save
- ☐ Delete own comment with confirmation
- ☐ Verify cannot edit/delete others' comments
- ☐ Test with multiple users and posts
- ☐ Verify real-time updates work correctly

Potential Enhancements

Future improvements could include:

1. Notification system for likes and comments
2. Reply functionality (nested comments)
3. Comment reactions or voting
4. Report/flag inappropriate content
5. Pagination for large comment sections
6. Rich text editor for comments
7. Mention system (@username)
8. Comment sorting options (newest, oldest, most liked)

Troubleshooting

Migration Issues

If migration fails:

```
# Reset database (development only!)  
npx prisma migrate reset  
  
# Or create migration without applying  
npx prisma migrate dev --create-only
```

Prisma Client Issues

If you see “Cannot find module ‘@prisma/client’”:

```
npx prisma generate
```

Type Errors

If TypeScript shows errors:

```
# Regenerate Prisma types  
npx prisma generate  
  
# Restart TypeScript server in your IDE
```

Support

For issues or questions:

1. Check the Prisma documentation: <https://www.prisma.io/docs>
2. Review Next.js API routes: <https://nextjs.org/docs/app/building-your-application/routing/route-handlers>
3. Check NextAuth documentation: <https://next-auth.js.org/>

License

This implementation follows the same license as the main project.